

Documentation: IonicModel.hpp

Implementation of the Bueno-Orovio Minimal Model for Cardiac Electrophysiology

Numerical Methods for Partial Differential Equations - Project 4

November 22, 2025

Abstract

This document describes the implementation of the `IonicModel.hpp` header file, which provides the ionic reaction term for the monodomain equation solver. The implementation relies on the *Minimal Model* proposed by Bueno-Orovio et al. (2008), designed to reproduce human ventricular action potentials efficiently.

Contents

1	Overview	2
2	Data Structures	2
3	Implementation Details	2
3.1	Constructor and Parameters	2
3.2	Total Ionic Current (J_{ion})	2
3.2.1	Fast Inward Current (J_{fi})	3
3.2.2	Slow Outward Current (J_{so})	3
3.2.3	Slow Inward Current (J_{si})	3
3.3	Gate Evolution	3
4	Code Structure	3
5	Conclusion	4

1 Overview

The `IonicModel.hpp` file implements the phenomenological model described in:

Bueno-Orovio, A., Cherry, E. M., & Fenton, F. H. (2008). Minimal model for human ventricular action potentials in tissue. Journal of Theoretical Biology.

This model is coupled with the Monodomain equation via an **Operator Splitting** scheme. The governing equation for the transmembrane potential u (dimensionless) is:

$$\frac{\partial u}{\partial t} = \nabla \cdot (D \nabla u) - J_{ion}(u, v, w, s) + I_{app} \quad (1)$$

where J_{ion} is the total ionic current density and v, w, s are gating variables evolving according to ordinary differential equations (ODEs).

2 Data Structures

The state of the ionic model at any given node (or quadrature point) is encapsulated in a lightweight structure:

```
1 struct IonicState {
2     double u; // Dimensionless transmembrane potential
3     double v; // Fast gating variable (depolarization)
4     double w; // Slow gating variable (repolarization)
5     double s; // Additional gating variable (sub-threshold)
6 };
```

Listing 1: IonicState structure

- **u**: Represents the scaled voltage, typically ranging from 0 (resting potential) to ≈ 1.6 (peak).
- **v**: Controls the fast inward current (similar to Na^+ channels). Range: $[0, 1]$.
- **w**: Controls the slow outward current (similar to K^+ channels). Range: $[0, 1]$.
- **s**: Adapts action potential duration and restitution. Range: $[0, 1]$.

3 Implementation Details

3.1 Constructor and Parameters

The class constructor initializes the model parameters corresponding to the **Epicardial** cell type (from Table 1 of the reference paper).

Key parameters include:

- Thresholds: $\theta_v, \theta_w, \theta_v^-, \theta_o$.
- Time constants: $\tau_{fi}, \tau_{si}, \tau_{so1}, \tau_{so2}$, etc.
- Characteristic values: u_o, u_u .

3.2 Total Ionic Current (J_{ion})

The method `compute_total_current` calculates the sum of three phenomenological currents:

$$J_{ion} = J_{fi} + J_{so} + J_{si} \quad (2)$$

3.2.1 Fast Inward Current (J_{fi})

Mimics the fast sodium current responsible for the upstroke. It is active only when $u > \theta_v$ and the gate v is open.

$$J_{fi} = \frac{v \cdot H(u - \theta_v) \cdot (u - \theta_v)(u_u - u)}{\tau_{fi}} \quad (3)$$

where $H(x)$ is the Heaviside step function.

3.2.2 Slow Outward Current (J_{so})

Mimics the potassium repolarization current.

$$J_{so} = \frac{(u - u_o)(1 - H(u - \theta_w))}{\tau_o} + \frac{H(u - \theta_w)}{\tau_{so}} \quad (4)$$

The time constants τ_o and τ_{so} are voltage-dependent.

3.2.3 Slow Inward Current (J_{si})

Mimics the calcium current responsible for the plateau phase.

$$J_{si} = -\frac{H(u - \theta_w) \cdot w \cdot s}{\tau_{si}} \quad (5)$$

3.3 Gate Evolution

The method `evolve_gates` updates the state variables v, w, s over a time step Δt . Since these variables do not diffuse, they obey local ODEs of the form:

$$\frac{dg}{dt} = \frac{g_\infty(u) - g}{\tau_g(u)} \quad \text{for } g \in \{v, w, s\} \quad (6)$$

The system is discretized using the **Forward Euler** scheme:

$$g^{n+1} = g^n + \Delta t \cdot \left(\frac{g_\infty(u^n) - g^n}{\tau_g(u^n)} \right) \quad (7)$$

This explicit scheme is computationally inexpensive and stable for the small time steps used in the simulation ($\Delta t \approx 0.005$ ms).

4 Code Structure

The class is designed to be modular and easily interchangeable.

```

1 class IonicModel {
2 public:
3     // Constructor: Initializes parameters (Epicardial set)
4     IonicModel();
5
6     // Computes the total reaction term J_ion
7     double compute_total_current(const IonicState &state) const;
8
9     // Updates gating variables (v, w, s) using Forward Euler
10    void evolve_gates(IonicState &state, double dt) const;
11
12 private:
13     // Model parameters
14     double u_o, u_u, theta_v, theta_w, ...;
15

```

```

16 // Helper functions for voltage-dependent time constants
17 double tau_o(double u) const;
18 double tau_so(double u) const;
19 };

```

Listing 2: Main methods of IonicModel class

5 Conclusion

The `IonicModel.hpp` implementation provides a robust and efficient solver for the reaction term of the cardiac electrophysiology problem. By strictly adhering to the dimensionless formulation of the Bueno-Orovio model, it ensures physical consistency when coupled with the normalized diffusion tensor in the Finite Element solver.